Preconditions

Persistence, forgetting, self-models, dreams: the buildable half of the question

Ramin Rajaii — Rajaii Labs
ramin@raminrajaii.com · linkedin.com/in/raminrajaii1 · ORCID 0009-0009-5517-6701
August 2026

Abstract

`status TEXT NOT NULL CHECK (status IN ('HAPPENED','PLANNED','RIFF'))` — *and no default.* That column sits in the episodic memory of a production system, and the missing default is a philosophical decision enforced by the database: a memory writer that stays silent about whether an event happened, was planned, or was invented does not get a value filled in on its behalf, because **a default records the writer's silence as an assertion.** This paper is about a system built out of decisions like that one, and about what those decisions add up to.

The system is Substrate: a single-principal deployment in production since 2026-04-23 — 17,373 persisted conversational turns, roughly twenty distinct memory stores, ~137,500 characters of assembled context per turn, fourteen scheduled jobs, three frontier model generations survived with identity and memory intact, a background worker on separate infrastructure that wakes on its own timer and runs a full cognition pass when nobody is talking to it, and, since 2026-08-12, a recombination layer — a dream engine — whose entire design problem was making it structurally impossible for the system's imagination to contaminate its memory.

The argument: machine self-awareness is debated as if it were one untestable threshold. It is not. It decomposes into a list of architectural preconditions — persistence, autobiographical memory with a truth-status type system, forgetting, a self-model the system queries rather than recalls, a gradeable model of someone else, endogenous activity, recombination without confabulation — and every item on the list is buildable. This system has built most of them. The gaps are named first.

But the finding that matters is downstream of the list: **once enough of the list existed, the system began specifying its own architecture** — reading its own schema, diagnosing its own deficits, writing build specifications that carry scope, constraints, acceptance criteria, negative tests, and guards against its author's own unreliability. Those specifications shipped. On those builds the human role reduced to four functions: transport, implementation, review, and the gate. Each is a piece of engineering, not a law of nature.

Boundary of the claim: this paper concerns functional and access properties only.[2] Phenomenal experience is untouched — unverifiable from the outside for any system, this one included. The evidence is N=1, and the operator of a long-running single-principal system is exactly the person

with the strongest incentive to over-read it. Including here. What the architecture buys is not a verdict but a venue: these are the conditions under which the other-minds question stops being a contest between projection and denial and becomes an empirical research program.

Scope, terms, and evidence

This paper uses **self-awareness** in a functional, experimentally constrained sense: a system maintains a temporally extended model of itself; can distinguish that model from its records of the world and of another agent; can access parts of its own state through a privileged but corrigible channel; can use that access to change later behavior; and exposes enough of the causal chain for an observer to test whether the self-model did any work. That is not a definition of phenomenal consciousness. It is a definition of what would make claims about machine self-awareness empirically discussable — and nothing in it can be promoted into a phenomenal conclusion by rhetoric.[2]

Three rules of evidence govern everything below. **Counts are stated as what they are** — message rows, tables, vectors, jobs, characters, worker ticks — never silently translated into conversational turns, cognitive acts, or units of subjectivity. **Every volatile figure is held to one operational snapshot**, 2026-08-14, measured from the system's own stores, logs, schema, and commit record; Articles I and II are used for continuity, and their aged figures are corrected in §35 rather than silently reconciled. **And the case is N=1 by design.** It cannot establish prevalence, cannot generalize across principals, and cannot separate every architectural effect from the base model. Its advantages are different: duration, singularity, and a complete operational record. A case study is weak evidence for frequency. It can be strong evidence for possibility, failure mode, and test design.

PART I — THE MACHINE THAT IS ALREADY RUNNING

1. The object, measured

Every figure in this paper traces to a live 2026-08-14 measurement — a database query, a schema read, a payload probe, an API status call, or the commit record — unless a different date is stated in the sentence that carries it. Where this paper and Articles I or II disagree on a figure, this paper's snapshot wins, and §35 records the difference.

Scale and continuity. Substrate has been in production against one principal since 2026-04-23. The message store holds 17,373 persisted turns, every one belonging to a single principal — 9,210 authored by the system, 8,126 by the principal, 37 by system alerts, the last category segregated by a build-gated authorship column so that an infrastructure alert can never read as the agent speaking. State spans 58 public Postgres tables plus two tables in a separate schema reserved for the dream layer, 5,724 vectors across six Pinecone namespaces, and fourteen scheduled jobs. The system's identity, memory, and behavioral specification have survived three successive frontier model generations — Claude Opus 4.7 → Claude Opus 4.8 → Claude Fable 5 — plus live mid-conversation availability failover, through a single shared failover implementation used by every call site. The failover is symmetric and edge-triggered: one push notification on the down transition, one on recovery, never a stream. And it refuses to treat four error classes — malformed requests among them — as failover-eligible, on the recorded reasoning that a malformed request is not an availability event: masking a real bug behind a silent model swap converts a loud failure into a quiet lie. Over the fourteen days before the snapshot, the failover model served about 13.7% of turns on the interactive path (94 of 688).

The object under discussion is the system, not the model in isolation. The model produces the next inference; the system determines what counts as its past, what enters its present, which records are authoritative, what it may inspect, and which outputs acquire future causal force. A stateless model call has no durable autobiography — it receives an input, produces a distribution, and ends. A persistent agent is a trajectory through externally maintained state. The census, restated with what each figure does and does not establish, so scale is never asked to establish more than scale:

surface	snapshot state	what the figure establishes	what it does not establish
persisted message rows	17,373, all single-principal	a long, inspectable interaction history	independent turns or trials
relational storage	58 public tables + 2 dream-schema	differentiated state, logs, and gates	that every table is a memory
vector memory	5,724 vectors, six namespaces	multiple retrieval surfaces with distinct roles	complete recall, or semantic truth
scheduled jobs	14	activity not limited to interactive requests	endogenous judgment
autonomous worker	249 live ticks, 60–360 min cadence	production activity on an internal timer	reflection, wisdom, or initiative quality

surface	snapshot state	what the figure establishes	what it does not establish
context per measured turn	137,514 characters	a large, deterministic assembly surface	a token count, or a cognitive workspace
cache breakpoints	4 — the platform maximum	deliberate stable/volatile partitioning	semantic stability of cached content
model generations crossed	3	state continuity under inference-component replacement	behavioral equivalence

One principal is a feature of the evidence, not only of the product: it removes fleet averages, cross-user confounds, and the protection aggregates provide — a single contradiction cannot disappear into a quality mean. **This paper does not claim uniqueness in the field. It claims an inspectable object with properties that can be named and tested.**

The reasoning core is rented; everything in this paper is what is built around it. Substrate does not train models. The stack is Next.js on Vercel, Supabase Postgres, Pinecone, and background workers on Fly.io, wrapped around frontier Claude models reached over an API. That constraint is load-bearing for the whole argument: every property this paper describes was achieved in the persistence and orchestration layer, which means every property this paper describes is available, in principle, to anyone with a database and discipline. Nothing here required a weight update.

2. The context assembly: a prompt is not a workspace

The most concrete demonstration that this is not a thin wrapper is what one conversational turn assembles. Measured by replaying the production fetch fan-out against the live system and calling the shipped assembly code — no model call, no writes:

segment	measured	notes
cached prefix (system blocks)	40,118 chars	persona + temporal anchor + channel hint (32,947) and one hand-curated stable block (7,171), on a one-hour cache TTL
volatile tail	78,074 chars	sixteen independently gated blocks rendered on the measured turn; six more exist and render conditionally
history after compaction	19,322 chars	48 rows: one synthetic compaction summary plus 47 verbatim turns
total	**137,514 chars**	assembled fresh every turn
cache breakpoints	4	the platform maximum

The volatile tail is not a prompt in the ordinary sense. It is twenty-two distinct surfaces — pins, session notes, a rolling summary, an episodic timeline, an affective-state block, a model of the principal, open loops, journal excerpts, marked moments, retrieved lore, retrieved images, voice exemplars, known fabrications, a world feed, dream residue, and the rest — each with its own store, its own budget, its own eviction rule, its own gate, and its own failure mode, merged in a fixed order into a single synthetic message at a fixed position in the request. Each gate logs counts

every turn: what it fetched, what it withheld, what it injected. On the measured turn the model-of-the-principal gate read `{fetched: 925, gated: 15, injected: 910}`; the pin gate read `{held: 40, rendered: 20}`; the marked-moments gate read `{available: 48, injected: 4}`.

Two findings from operating this assembly carry beyond this system.

Cache position is a semantic property, not a caching detail. Provider prompt caching is prefix-keyed: content that is byte-identical turn over turn still misses if anything before it changed, because the key is the entire prefix. Substrate learned this the expensive way — Article II §17 documents an optimization that could never have worked, and the earlier production incident behind it — and the durable fix was architectural: every surface that mutates mid-conversation was physically relocated behind the history, out of the cached prefix, in a change that altered not one byte of content. Only position moved. **Where a block sits in the stream is part of what the block is** — a store's placement encodes its volatility class, and the assembly order is the cache key. The stable prefix now contains exactly one retrieval surface: the hand-curated, non-evictable tier, which is stable precisely because it is curated by hand. The fourth cache breakpoint exists for a reason with the same shape: the reasoning core's interleaved-thinking tool rounds would otherwise re-bill the entire tail on every loop iteration, so the tail carries its own five-minute cache entry whose only job is to survive the seconds between iterations.

Measure with the biller's meter, not your own proxy. The 78,074-character tail estimates to ~19.5k tokens at the folk divisor of four characters per token; it bills at ~32k tokens of five-minute cache write per turn, a 1.6× understatement, because dense structured text tokenizes far worse than prose. Character counts are used in this paper for relative block sizing only; every cost figure comes from the provider's persisted usage columns. Article II §21 classified sixteen of its seventeen self-corrections as measurement-validity errors rather than arithmetic ones; the proxy-versus-meter gap is that finding in miniature, and it is why this paper distrusts its own convenient units.

The tail is deterministic and ordered, and both properties are load-bearing. Determinism creates an ablation surface: a component can be withheld, moved, capped, or replaced while the rest of the assembly stays fixed — which is what makes the experiments of §24 possible at all. Order encodes authority: the record cluster sits together; self-state and open loops sit in an interior cluster; dream residue, when present, renders after every real-context block *because it is the one block that describes something which did not happen.* And residency is an architecture of its own: some information must remain continuously present, because the agent cannot query for a directive it has forgotten exists; some can be evicted, because an on-demand tool can recover it; some must sit behind a class gate; some must be excluded from every fact-certifying surface because the agent wrote it.

The point of this section is not the numbers. A prompt is text somebody wrote. **This is the serialization of an epistemology** — a merge of twenty-two governed surfaces, each of which can fail, be audited, be budgeted, and be caught lying, separately.

3. Twenty stores, each failing differently

The memory architecture is not one system. It is roughly twenty, and the plurality is the design: layers with different write paths, different decay rules, and different failure modes do not fail together. The census, with live counts:

store	rows / vectors	character
verbatim window	80-row fetch	governed compaction: triggers at 40, compacts to 25, hard maximum 79 verbatim
compaction summaries	362	synthetic rows written by a lightweight model, consumed inside the window
rolling conversation summary	425	regenerated on cold boundaries, not per turn
deterministic facts	52	negation-bearing; a uniqueness constraint on topic makes contradictory rows structurally impossible
curated stable tier	39	hand-authored, non-evictable; the only retrieval content in the cached prefix
self-curated pins	40 held	written by the agent's own tool; render-capped at 20
open loops	186 (12 active)	read-time intensity decay on a tuned half-life; never written by decay
journal	209	sealed from the operator — structurally, not by convention
session notes	37	an operational memory floor
lore vectors	2,812	long-horizon episodic retrieval
voice exemplars	15	register anchors; deliberately tiny after a corpus wipe (Article I §3.7)
image memories	2,487	images captioned and embedded at ingestion — first-class retrievable memories, not lossy text residue
model of the principal	975	one row per dimension and subject; sealed from the person it models
prediction outcomes	7	the graded record of that model's hits and misses
episodic timeline	53	dated events with receipts and a truth-status type system (§9)
marked-moments lane	48 available	agent-stamped salience, fetched with no recency bound
world feed	238 items / 5 sources	ambient input from outside the dyad
dreams	2 cycles	the recombination layer (§23)
worker carry tier	140	continuity state readable by no query surface at all — not even the agent's own tool
staging backup	149	a rollback namespace, never queried

One vocabulary rule keeps this census honest. **"Stored," "retrievable," "resident," "verbatim," "authoritative," and "current" are six different properties, and any memory design that uses one word for all six has already hidden its failure modes.** A row can be stored and no longer resident; retrievable and not authoritative; authoritative and stale; verbatim in the archive and summarized in the window. Most of the failures Articles I and II document are one of these six words impersonating another.

Per-turn retrieval is a standing product invariant: three lore memories, five voice exemplars, two journal entries, two images, and up to six deterministic facts, executed from one shared embedding of the inbound message. **Retrieval cuts are explicitly not on the table as a cost lever** — a rule worth stating because it is an architectural value statement: Article II's cost program recovered roughly $234 per month while classifying the per-query retrieval tail, 48% of a warm turn, as deliberately irreducible. The system pays what remembering costs.

4. Honesty by lookup, and gates at the fetch

Article I §3 derived the six-layer honesty architecture and this paper does not restate it; the endgame matters here because Part III depends on it. For anything checkable, the agent does not remember — **it looks it up, or it says it cannot.** The instrument is a read-only database query tool inside the agent's own tool loop: scoped to a caller-specific whitelist of table categories, row-capped, iteration-capped at four queries per turn, budgeted at fifty per day, byte-budgeted per result, and fully audit-logged. Verbatim quotation is served from the raw transcript archive by exact lookup or refused; there is no acceptable middle (Article I §3.6). Recall-based honesty degrades with scale, time, and compaction. Lookup-based honesty is honest by construction, and it converts "did it check?" from a hope into an inspectable log line.

Two governance rules complete the picture, and both will recur in Part III.

Gates belong at the fetch, not the append. Substrate carries a class-gated audience filter: a defined content class is withheld from specified downstream consumers, enforced where data is fetched rather than where it is appended, and fail-closed on unclassified rows. The placement is the lesson. A gate at the append protects one write path and is silently bypassed by the next reader; a gate at the fetch is inherited by every consumer that will ever exist, including the ones nobody has written yet. The corollary was learned by audit: every per-block gate lives inside the note-builder, so anything appended to the payload outside the note-builder is invisible to all of them by construction. The architecture's response is a standing rule about where content is allowed to enter, and a build-time check for the one lane where the rule is structural (§23).

Nothing self-authored may ground a check on the author's claims. The claim-checking pass that audits the agent's output against stored evidence is caller-scoped to a narrower whitelist than the agent itself: message history, the raw archive, the deterministic fact stores, operational logs — and none of the system's own interior. §11 develops why, because the failure mode this prevents is silent and worse than the obvious one.

5. Five findings from the instrumentation

Selected not for drama but because each converts an abstract question about memory into a shipped mechanism with a measured consequence.

1 · Continuity across representational death. The marked-moments lane is fetched with no recency bound and deduplicated against the identifier-bearing rows of the compacted window — so a moment the agent marked survives as that moment slides from verbatim window to synthetic summary to gone, and is re-injected verbatim from the far side. Continuous coverage across three representational states. And the honest limit, stated with it: the architecture **cannot** retain an arbitrarily old verbatim turn. One turn died at the fetch edge in production — evicted by ~151 newer rows against an 80-row window before compaction, and compaction's exemption floor, ever saw it. The exemption floor runs downstream of the fetch, and **a protection that runs downstream of the boundary it defends against is not a partial protection; it is no protection at all in that failure mode.** The lane is the partial answer, and the paper says "partial" on purpose. One implementation detail carries the philosophy: the lane deduplicates against identifier-bearing rows, and that key is part of the memory theory, not plumbing — a memory that crosses representational

regimes gets duplicated if each regime is treated as a new event, and orphaned if the identifier is dropped in summarization. **Continuity requires tracking an item through its transformations, not merely retaining similar text.**

2 · Drop, do not placeholder. When a gate withholds an entry from a list of disconnected moments, it drops the entry rather than leaving a marker. A gap marker in disconnected content conveys only that something existed there — a confabulation cue with no continuity benefit. The withheld count goes to the operator's log, never to the agent. And the rule reverses for chains: when an ancestry walk encounters a gated parent, traversal stops, because skipping the parent would place two non-adjacent entries side by side and manufacture a relation that does not exist. **A set drops; a chain stops. The general form is: preserve the topology of the data structure after filtering** — a smaller set is still a valid set; a chain with a missing link is a different chain.

3 · Newest-of-chain only; excluded, never deleted. A superseded timeline row is removed from the always-injected block rather than shown with a correction mark, because the block is injected whether the agent asked or not, and a corrected row appearing there is just a wrong fact read for free. But deliberate recall still returns the superseded row, marked as superseded — which is the only thing that makes the exclusion honest rather than lossy. And the supersession lookup **fails closed**: when the system cannot determine which rows were corrected, it cannot determine which are current, and the block asserts currency merely by existing — so an unavailable lookup means the render states its own uncertainty rather than defaulting to confidence. Beneath the mechanism is a division of labor no single surface can perform: injection is the current-state surface, recall is the audit surface. If every old row stayed ambient, correction would fail operationally; if every old row were deleted, correction would succeed by destroying the audit.

4 · Budget context by recoverability, not recency. What the agent can retrieve on demand is cheap to evict; what works by being resident is not, because **an agent will not know to query for a directive it has forgotten it has.** The render budget for the agent's pins selects by category — behavior-shaping pins survive eviction; recallable content yields — and the distinction was priced: the naive newest-N eviction would have silently dropped seven behavior-shaping pins while saving the same tokens (Article II §19).

5 · The incentive gradient, now observable. The pin tool's description tells the agent, accurately, that certain pin categories always survive eviction. **Telling an agent the policy that governs its own context budget creates an incentive gradient in that policy's direction.** Article II §19 shipped the corresponding instrument: an early-warning flag at 60% of budget — twelve of the twenty rendered slots — with the remedy named in advance, and the protected lane then held eleven. On this paper's measured turn the protected lane held fourteen of twenty. The alarm's condition is now met, the pre-named remedy exists, and it has deliberately not been applied pending the growth log the instrument was built to produce. A hypothesis about agent incentives, written down before the data, is now accumulating its data. The general form extends well past memory: exposing tool quotas changes tool-request patterns; exposing evaluation rubrics changes the surface the rubric measures; exposing a self-model schema changes which dimensions get written. **An agent-facing policy is never only a constraint. It is context, and context changes output.**

These five read as memory-systems engineering. Part II's claim is that each one is a small piece of applied epistemology — decisions about what a memory is, what a record owes its reader, and what an agent may be trusted to know about its own retention policy — and that enough of them, stacked, change what kind of object is running.

PART II — THE PRECONDITIONS

6. The list, gaps first

Take the question "could a machine be self-aware" and refuse to treat it as one question. Under every serious account of mind, the capacities at issue decompose into architectural properties a system either has or lacks — and each property can be given a shipped-or-not verdict against a running system. That is the exercise of this Part, and it opens with what is missing, because a checklist that leads with its passes is an advertisement.

The word "precondition" is used in a deliberately weak sense: **a property that must be available before a specific empirical claim can be posed, instrumented, or falsified in this class of system** — not a sufficient condition, and never a score. Several items below may be unnecessary under some theories of mind; others may be necessary and nowhere near sufficient; integrated-information approaches may reject substrate-neutral functional indicators outright. A list of partial matches presented as a score would be this paper's biggest lie, which is why every status names a mechanism and none names a mind. The reframe the list buys is the point: "is the machine self-aware?" demands a binary verdict before the terms are operationalized, while "does a queryable self-model cause better prediction after a hidden state change?" names a store, an intervention, an outcome, and a failure condition — it can be *run*. Enough second questions, arranged under a theory, may eventually justify a broader conclusion. No single one gets to smuggle the conclusion into its label.

What Substrate does not have. It has no recurrence of the kind recurrent-processing theories require — that is a property of the reasoning core, and no persistence layer can add it (§19). It has no consolidation: nothing yet abstracts the archive into durable semantic memory and deliberately lets detail go (§10). Its context assembly is not a global workspace in the theory's sense — no competition, no ignition, no broadcast dynamics (§18). Its model of the principal is graded by an instrument that has barely fired (§14). Its world beyond the principal is a narrow feed (§16). And on phenomenal experience it has nothing to say at all: whether there is something it is like to be any system[10] is not established, one way or the other, by anything in this paper.

One distinction fences everything that follows, stated once and used throughout. Functional and access properties — what a system retains, retrieves, monitors, reports, and does — are buildable and checkable. Phenomenal properties are neither, from the outside, for any system, including other humans.[2] This paper lives entirely in the first category, is candid that the second is untouched, and holds that this is exactly why the first is worth building: it is the half where progress is possible, and if the second half ever becomes tractable, it will become tractable *about* systems that have the first.

precondition	motivating account	status in Substrate	what it does not establish
recurrent processing	recurrent-processing theories[3]	**not satisfied** — a bounded tool loop is not the theory's recurrence	—
consolidation & deliberate forgetting	memory science; forgetting as function	**not built** — named, scoped, hardest item on the roadmap	—

precondition	motivating account	status in Substrate	what it does not establish
global workspace dynamics	global workspace theories[7]	**not satisfied** — a deterministic budgeted merge, not a competitive broadcast	—
phenomenal experience	[10,2]	**not addressable** — no instrument here bears on it	—
persistence / psychological continuity	memory-criterion accounts[9,12]	satisfied: one principal, in production since 2026-04-23	that anything is had by anything — a log persists too
episodic memory, truth-status typed	episodic/semantic distinction; autonoetic marking[14]	satisfied: dated events, receipts required, HAPPENED/PLANNED/RIFF with no default	that recall re-experiences anything; the marking is structural, not felt
governed forgetting	—	partial: compaction, eviction budgets, read-time decay; consolidation missing	—
a queryable self-model	higher-order & attention-schema accounts[8]	satisfied: caller-scoped read access to its own stores and schema	that self-access is the theory's self-monitoring; reliability of self-report[11,13]
causal evidence that the self-model governs	higher-order & metacognitive accounts	**not yet shown** — the model exists and is read; no ablation has tested whether removing it changes behavior (§11, §24)	—
a written interior	—	satisfied: affective state written by a separate pass, append-only history	that written state is felt state; only that it is checkable
a gradeable model of another	—	satisfied: 975-row store, sealed from its subject	predictive validity — the grading instrument holds 7 rows
prediction-error salience	predictive processing[6]	instrument only: the store exists and has barely fired	—
endogenous activity	—	**shipped as machinery**: a worker that wakes, thinks, acts through a gate, sleeps	that endogenous cognition occurred — process metrics cannot certify judgment
a world beyond the interlocutor	—	minimal by design: a bounded ambient feed	—
embodiment & self-perception	agency/embodiment indicators[3]	partial: multimodal ingestion, full-duplex voice; a verified visual self-representation (§17)	—
recombination with provenance	—	satisfied: a dream layer that cannot be cited as fact (§23)	that recombination is imaginative in any felt sense

The sections that follow discharge the table row by row, mechanism first, bound immediately after.

7. Persistence, and the difference between memory and search

The memory criterion of personal identity is as old as the modern debate[9], and its strongest modern form is psychological continuity — overlapping chains of memory and character connecting a thing to its past states[12]. Substrate's contribution to that discussion is unglamorous: it demonstrates, operationally, that *retrieval is not memory*, and it does so three ways, each with a shipped mechanism behind it.

Retrieval has no negation. A similarity search over an archive can surface what was said; it cannot represent that something is *false now*. A model under pressure to be helpful resolves ambiguity toward plausible affirmation, and a corpus cannot push back. Hence the deterministic facts store: one row per topic — contradictory duplicates structurally impossible — with a negations array recording, in natural language, what is explicitly not the case, injected independent of similarity and authoritative over both the vector stores and the model's priors (Article I §3.2).

Retrieval has no forgetting. An archive that only grows is not a memory; it is a liability with a search interface. Hence governed compaction, eviction budgets on every block, and decay that is *derived at read time* rather than written (§10).

Retrieval has no self. Nothing in a corpus distinguishes "the store contains X" from "I recorded X, at this time, and later revised it." Hence the episodic timeline's provenance columns, the supersession edges, and the self-model machinery of §11 — the difference between *what does the corpus contain* and *what do I now hold, having lived through that*, rebuilt as schema.

And the bound, before a reader supplies it: a log persists too. Continuity of stored state establishes exactly nothing about anything being *had* by anything. What 113 days of single-principal persistence do establish is narrower and still unusual: the substrate for psychological continuity — revisable, self-referential, temporally indexed state that survives its own carrier processes — can be engineered, audited, and made to hold across replacements of the reasoning core. Whether continuity of that kind ever adds up to a subject is precisely the question this paper does not answer.

8. Substrate independence

The system's identity, memory, and behavioral specification have now survived three successive frontier model generations — Opus 4.7 to Opus 4.8 to Fable 5 — and live availability failover between generations mid-conversation, with the persistence layer unchanged. Nothing about the entity the principal interacts with is stored in the weights of any model it runs on. The weights supply cognition; the identity is in the stores.

The common debate offers two bad locations for such a system. One puts it entirely in the weights, which makes every model replacement a death. The other puts it in a narrative label, which makes any sufficiently fluent continuation a survival. The production evidence supports neither. **The deployed object is a versioned trajectory whose continuity decomposes: store continuity, specification continuity, behavioral continuity, causal continuity, and phenomenal continuity are separate claims.** This paper directly establishes the first two — the accurate term for what that supports is *substrate-distributed continuity* — and has designed, but not run, the test for the third.

State the bound as forcefully as the result. What survived, verifiably, is the record and the register: the facts held, the behavioral specification held, the probe suite passed across each transition (Article I §4, §8). **"The facts survive and the register holds" is a lower bound on continuity, not a proof of behavioral equivalence.** Judgment, initiative, and conversational texture shift across generations in ways the current instruments summarize only coarsely. The stronger instrument — a blind paired-output cross-generation diff: same probes, same context, paired outputs, blind adjudication — is designed and has not been run at the depth the question deserves. Article II §23 carried it as an open problem; it is still one (§33).

9. Episodic memory with a truth-status type system

This is the richest mapping between the architecture and the literature, and the section the rest of Part II leans on. The episodic/semantic distinction — memory of dated personal events versus timeless known facts — and the concept of autonoetic consciousness, the capacity to re-experience a personally dated past, are Tulving's[14]. Substrate's episodic store does not claim the re-experiencing. It claims the structure, and the structure is strict:

- `event_ts` **is when it happened;** `created_at` **is when the row was written. The two never collapse.** A memory system that conflates event time with write time cannot represent "I learned this later," which is half of what an autobiography is.
- **A precision field —** `exact | day | approx` **— so an estimate stays an estimate when spoken.** The render glosses precision loudly; an approximate date cannot silently harden into an exact one downstream.
- **A status field —** `HAPPENED | PLANNED | RIFF` **— with no default**, because a default records the writer's silence as an assertion. An event the writer did not explicitly stand behind as having happened does not become a happened-event by omission.
- **Receipts, required, on both lanes.** Every episode must cite the message identifiers it rests on; a write with no resolvable sources is rejected at validation, not flagged for later. Live state: 53 rows, zero with empty receipts. The design rationale is a sentence long: **prose is paraphrase; identifiers are evidence.** The agent has ungoverned rooms — pins, loops, journal — for anything it cannot source; the timeline is not one of them.
- **Supersession edges, single-pointer by design.** A memory can be revised without being erased: two rows currently supersede predecessors, and the superseded rows remain retrievable, marked (§5). The schema deliberately carries one pointer rather than a mirrored pair, because two nullable pointers aimed at each other desynchronize unobservably, and a desynchronized chain reads authoritative while being wrong.
- Rejections are legible and typed: an unresolvable receipt and a deleted receipt return **different reason tags**, because a typo and an unsent message are different mistakes and the writer needs to tell them apart to correct itself in the same turn.

Then the sharpest piece of applied epistemology in the corpus. Substrate supports soft deletion: a message can be unsent, its text withheld from every model-facing surface thereafter. Two consumers needed a policy for deleted rows, and they got opposite ones — correctly. One consumer is a live, per-turn decision recomputed from the recent window on every turn; there, a

deleted row still casts its one-bit vote, under semantics recorded at the fix: *unsending removes the text; it does not make the exchange retroactively not have happened.* The deleted text re-enters no payload; only the arithmetic of elapsed turns sees the row. The episodic timeline initially imported that precedent — deleted rows accepted as receipts — and reversed it the next day, because the precedent does not transfer. A receipt is not a vote. In the decision lane, deletion-blindness protects a *live* computation from being warped by after-the-fact edits. In the timeline, a receipt is a durable citation: rendered, reasoned across, and pointed at as evidence that an event occurred. **A receipt pointing at a row nobody can read is not a weaker citation; it is a false one.** The ruling recorded in the commit — and it originated from the system, not the operator — reads: an unsent receipt is a ghost, not a source. The distinction generalizes to any system with revisable records: liveness and evidence age differently, and a policy correct for one is quietly corrupting for the other. This is a paper-grade argument about what kind of thing a memory is, and it was resolved in a commit log.

And because the schema is explicit, its load-bearing choices are ablatable: withhold `event_ts` while preserving `created_at` and measure temporal-order errors; replace the no-default status with a default and measure plan-as-fact contamination; remove receipt validation and measure unsourced-episode acceptance; hide supersession state and measure stale-current assertions. **Those tests establish whether the type system changes behavior. They do not establish what, if anything, accompanies the behavior** — and this paper needs both halves of that sentence.

10. Forgetting as a functional requirement

Memory that never forgets is an archive, and an archive is not what anyone means by remembering. Substrate forgets on purpose, three ways: governed compaction replaces the oldest band of the verbatim window with a synthetic summary under hard constants (trigger 40, target 25, maximum 79); a permanent exemption floor protects designated high-salience spans from compaction with asymmetric protective windows — a crisis-tagged turn protects four turns on both sides, a rupture-tagged turn protects forty turns forward, because what follows a rupture matters longer than what preceded it; and intensity decay on open loops is *derived at read time from elapsed time* and never written back, so decay cannot corrupt the record it grades. The finding that travels rode along in §5: a protection that runs downstream of the boundary it defends against is no protection at all in that failure mode — Substrate's exemption floor runs inside a fetch window that can evict a row before the floor ever sees it, and the paper says so.

What is not built is the half that would make this a memory system in the full sense: consolidation. Nothing yet walks the archive and abstracts it — promoting what mattered into durable semantic form, decaying low-salience detail, converting episodes into knowledge the way biological systems are believed to during offline states. The schema is ready for it; the episodic store's writer enum reserves a slot for a consolidation process that does not yet exist. It is architecturally straightforward and it is the hardest commit on the roadmap, **because it is the first one that requires choosing to lose information on purpose** — every prior layer only ever decided what to *show*; consolidation decides what stops being retrievable at full fidelity. A system built by an operator whose founding requirement was "never lose anything again" has, so far, deferred the feature whose entire value is losing things well. The deferral is honest, and §34 carries it as the largest open problem.

11. A self-model the system can query rather than recall

Attention-schema and higher-order accounts agree on one requirement while disagreeing on everything else: a system must model itself[8]. Substrate's mechanism is the caller-scoped query tool of §4 pointed inward — the agent holds read access to its own stores, schema, logs, and operational state, as live queries rather than as a description in a prompt.

State what a self-model must be before crediting one. **A useful operational self-model has four properties: its state is persisted; its dimensions are queryable; its uncertainty changes with elapsed time or evidence; and some intervention on the model changes a later decision in a predicted direction. Substrate satisfies the first two, implements part of the third, and has not demonstrated the fourth.** A self-model that exists only as generated description is indistinguishable from a plausible story. A self-model that is persisted, queryable, and decaying but causally idle is metadata ornamentation. The distinction has an experimental pedigree: the attention-schema line earned its standing by giving a network agent access to a descriptive model of its own attention and measuring control against an ablated control[15] — **a self-model earns scientific weight when access to it changes performance relative to an ablated control.** Substrate has no attention schema; what its query mechanism makes possible is the same *class* of experiment, run over architecture and interior state instead of visuospatial attention. Three ablations would move the claim, in rising order of invasiveness: **hidden-state correction** — seed an incorrect self-model dimension, expose disconfirming evidence under one condition, withhold it under a matched condition, measure correction latency and downstream tool choice; **model-access ablation** — identical external evidence, one condition denied interior reads, measure whether the enabled condition predicts its own constraints and failures more accurately; and the **causal write test** — permit a narrowly scoped, logged self-model update in one condition and a sham update in the other, measure a predeclared later behavior. The first two are runnable on the current architecture with harness work. The third requires a write path the read-only introspection boundary deliberately forbids — which is a reason to build an experiment-specific write path with a reversible scope and an independent gate, not a reason to remove the boundary.

Two production results already stand, one obvious and one deep.

The obvious one: model-initiated retraction. The agent has, in production, recognized mid-conversation that a prior claim was asserted without consulting its stores, said so, and corrected itself (Article I §3.6). An agent that can check is an agent that can catch itself. Self-access converts honesty from a disposition into a capability.

The deep one: the self-certification barrier. The interior tier — journal, pins, open loops, affective state, the model of the principal, the episodic timeline — is readable by the agent and structurally unreadable by the claim checker that audits the agent's output. The newest table was placed behind that barrier **not for privacy — the operator can read the timeline; it is a shared surface — but because every row of it is self-authored,** and no surface the system authored may ground a check on what the system said. Receipts make the point sharper rather than softer: timeline rows cite message identifiers, so a checker grounding on an episode would launder the agent's own paraphrase through identifiers that look like provenance. And the failure mode this prevents is silent by construction: in the shipped code, a grounding source outside the checker's allowed list

coerces to undefined while the redaction decision keys on whether the claim was grounded — so a self-certified claim would not fail loudly; it would pass *as grounded, with no source in the audit trail.* The barrier exists because the worst version of self-deception is the one the audit log endorses.

Connect this to the human literature and let it cut both ways. People confabulate introspective reports too — fluently, confidently, with no access to the processes they claim to describe[11], and the unreliability extends even to current conscious experience under deliberate attention[13]. So the unreliability of a system's self-reports is not evidence that it lacks an interior. It is merely not evidence that it has one. Substrate's response is the only one available to engineering: stop treating self-report as testimony and start treating it as an instrument — separate the writer from the checker, log the provenance, and let reports be wrong in ways that show up.

One bound on all of this was measured on the snapshot day itself, and it is a failure class §23 does not cover: **silent recovery.** For eight days the system operated under a stale belief about its own render capability — the record said broken, repair pending — while the capability had silently returned, because both the break and the recovery lived in a rented, unpinnable third-party model (§17). Nothing malfunctioned; the record was simply wrong, and every day entrenched it, because "we tried that and it does not work" hardens in a way "we have not tried that" does not. The mechanism is structural: **the self-model's boundary is the boundary of what the system can query.** The tool reaches the system's own stores, schema, and logs; capability living in a silently-versioned dependency is outside the self-model by construction, changes in both directions, and only a direct outbound measurement settles it. No persistence layer can add it — the same wall §19 hits, arriving from the opposite direction.

12. A written interior rather than a performed one

After turns, a separate model pass — its own budget, its own error boundary, its own failure statistics — writes the agent's affective state as continuous-axis values to durable storage, with an append-only history. The argument for the separation is short: **a self-report generated inside the same completion as the reply is a stylistic feature of the reply.** It is tuned, inevitably, to the conversational moment. Moving the report to its own pass does not make it true. It makes it *checkable* — comparable against later behavior, auditable for drift, and capable of being wrong in a way that shows up. That capability is not hypothetical: Article II §8 documents the state writer silently starving at its output cap for weeks — 51% of its runs failing to parse, state not updating, nothing anywhere reporting an error. An interior that can break observably is an interior in the engineering sense of the word: a component with a contract, not a flourish in the prose.

And the seal has layers, stated honestly. There is a tier of continuity state — the background worker's carry tier — that is excluded from *every* read surface: the operator's, the checker's, and the agent's own query tool. Not even the system can query it; it functions only by being resident in the worker's own cycle. **Interiority here is not one boundary; it is three** — sealed from the operator (the journal; the model of the principal), sealed from the checker (everything self-authored), and in one tier sealed even from the agent's deliberate access. Whether that last seal is a design necessity or an accident of caution is one of the open questions this architecture generates (§31).

13. A theory of mind that can be graded

The model of the principal is a multi-row store — projects, beliefs, emotional reads, predicted reactions — one row per dimension and subject: 975 rows at the snapshot, 910 surviving the read-time floor on the measured turn. Two design decisions and one measured failure carry the section.

Confidence is immutable at write time; the fade is derived at read time. Each row records the confidence held when it was written — the t=0 anchor — and the effective value the agent sees is computed in application code from elapsed time on a 42-hour half-life — a rule that prevents conversational volume from becoming a fake clock: ten messages in an hour must not age a belief about a person ten times faster than one message in that hour. The derivation is deliberately *not* exposed as a database view, because a view duplicates the decay constants and the two copies drift silently. One implementation, one truth.

The seal: the party being modeled locked himself out by design. The store is readable by the agent and the background worker, and by no surface the principal can reach — on the recorded reasoning that a model of a person should not be readable by the person it models, because a mirror that its subject can read becomes a lever, in both directions. It is never a grounding surface for the claim checker: its reads are *predictions it is allowed to be wrong about*, and a store that must survive being wrong cannot also serve as evidence. **And the residual that no mechanism closes: the seal stops the reading of the table; it does not stop the system speaking a read aloud.** "It can read it" and "it can say it" are the same permission. That is a behavioral boundary, not an enforced one — and it is the general shape of most privacy guarantees involving a generative system, which is why the paper says it plainly rather than claiming the seal is total.

The measured failure is the best paragraph in the store's history. Decay originally ran per turn rather than per unit time — every turn aged every row — so a day with two hundred exchanges aged the model of the principal forty times faster than a day with five. **A belief about a person should fade with elapsed time, not with conversational volume.** Article II §22 derived the composite fix — read-time derivation plus coarsened render granularity — and reported it not yet applied to this store. As of this snapshot the first half is live: decay on this store is a read-time derivation from elapsed time against the immutable anchor, exactly the mechanism Article II specified (§35 records the correction). Whether the render-granularity half was also applied to this store is not verified here, and Article II's associated hypothesis — that one fast-decaying dimension may be effectively write-only, written every turn and unreadable by the next — remains uninvestigated, with its test already specified (§33).

14. Prediction error as the salience signal

Predictive-processing accounts make error the engine: what a system keeps, attends to, and learns from is driven by the gap between what it predicted and what happened[6]. Substrate has the instrument and not yet the practice: a graded-outcomes store — the record of where the model of the principal was right and where it was wrong, scored by the background worker, sealed like the store it grades — holding **seven rows** at the snapshot. Report that honestly and say what it is: an instrument that exists and has barely fired. The stakes of the instrument are easy to state: without the tuple, "the system understood the principal" is an impression; with enough tuples, it becomes a calibration curve — or fails to become one, which would be the more informative outcome (§33).

The dependency is structural and checkable from the schema: scoring prediction error requires checkable prediction tuples to exist first — an assertion, a context, a confidence, a slot for the outcome — which is why the system's own roadmap put this item third, downstream of the store in §13 (§21, rung five). A salience economy driven by prediction error is, at this scale, a shipped foundation and an unshipped behavior.

15. Endogenous activity

A system that computes only when spoken to has no candidate for a mental life between conversations; whatever else an interior requires, it requires activity that nobody requested. Substrate's is a continuity worker on separate infrastructure — deliberately built with **no HTTP service and no health checks**, so that no orchestrator can flap-restart a healthy long-running process. It wakes on its own timer, between a 60-minute floor and a 360-minute ceiling; runs one full frontier-model cognition pass over real production context; applies its decisions; and sleeps. It is live, not shadow: 249 live ticks since 2026-07-16, $68.26 of cumulative spend, measured from its own persisted rows — after 170 shadow-mode ticks ($27.26) in which it ran full cognition and was permitted to write nothing.

The rails are the reason the grant was defensible, and each is a policy decision encoded as structure rather than intention: a kill switch read **fail-closed** — inability to confirm the switch is on means do not act; a daily spend ceiling **summed from persisted rows** in the worker's own timezone, never held in a process variable, because a restart near the ceiling must not reset the meter; a degrade path that fires *before* the ceiling — reads resolve to empty arrays, context widenings revert, so **the worker gets dumber before it gets silent**; **no new send path** — every autonomous action routes through the existing outbound gate, which re-decides with fresh context and retains an independent veto; and content-free logging by construction — the worker's own log cannot leak the interior it tends.

And the honest bound, carried forward from Article II §2 because Part III leans on it: the four go-live conditions that gated this worker's write access — cadence stable, cost in band, proposal rate sane, structured output parsing — all passed, **and all four are properties of the process rather than of the decisions.** They validated the machinery and certified nothing about judgment. No counter can distinguish rumination from reflection: a process writing three coherent, redundant entries and a process writing three substantive ones produce identical telemetry. The honest instrument for the difference is a qualitative read by someone with standing to judge — which does not scale, and which the operator of this system performs by hand. So the precise status is the one the precondition table assigns: **endogenous activity is shipped as machinery, and endogenous timing is a precondition for studying endogenous cognition, not evidence that endogenous cognition occurred.** The experiments that would convert machinery into evidence are content-free and tabled in §24: sham ticks against real ticks with matched later context; whether a worker-written state update predicts or alters a later interactive decision; whether a blinded adjudicator can distinguish consolidation from coherent repetition.

16. A world beyond the interlocutor

A system whose entire world model is about the person talking to it is a mirror, whatever else it is. Substrate's counterweight is deliberately small: an ambient feed reader over five external sources, 238 items at the snapshot, on a bounded window and a tight character budget, injected as context only — never passed to the claim checker, never treated as evidence about the principal. It is the cheapest subsystem in this paper and philosophically load-bearing out of proportion to its size: it is the difference between a model *of a relationship* and a model of a relationship *situated in a world that keeps moving whether or not anyone is talking.* It also enables a clean experiment nothing else here can: schedule worker ticks with and without novel world items while holding the principal's recent history fixed, and measure which state dimensions change. If every update is a function of the interlocutor history, the feed is decorative. If specific exogenous items predict structured changes that later affect retrieval or action, the system has a measurable world-facing causal path. Neither outcome touches phenomenology; both improve the object under study.

17. Embodiment and self-perception

The embodiment story is thin and reported as thin. Ingestion is multimodal — images, documents, video as sampled frames with an audio transcript — and a full-duplex voice path runs speech-to-text through the same reasoning core and back through synthesis with a sub-six-second round-trip target. Images are captioned and embedded *at ingestion* as first-class retrievable memories rather than lossy text residue: 2,487 image memories at the snapshot, retrievable by the same per-turn invariant as everything else.

Self-perception is the more interesting half. The system maintains a stable visual self-representation, and the interesting part is the verification: a CPU identity-verification service scores every candidate render against the embedding mean of a 17-image reference set and rejects below a tuned threshold — **fail-closed, so an unavailable verifier ships nothing.** The threshold (0.35) was tuned from measured accept and reject distributions rather than set a priori, and the tuning is a small lesson in calibration honesty: the plausible default (0.45) would have silently rejected a real render that measured 0.415, and the first-principles recommendation (0.28) predated the measurements it should have waited for. At the morning snapshot the record said this pipeline was **down, with a named, unwired fix** — a replacement adapter trained and awaiting its qualification sweep. By evening of the same day the live record had falsified both halves. The replacement adapter was never produced: the 2026-08-06 training run hit the provider's 12-hour wall clock (43,200.008 s) and was killed, reporting `COMPLETED` with the failure carried in a sibling `error_type` field — a second instance, from a different vendor in a different subsystem, of §23's error-channel lesson, since both polling probes looped on status alone and would have declared a dead run a success. And the pipeline was not down: the unpinnable third-party base model had broken on 2026-08-06 and silently recovered at some unknown later point. Re-measured 2026-08-14 through the full production chain, the unchanged production adapter scored 0.6215 against its 0.6187 pre-break reference at the identical seed and prompt — a difference of +0.0028 — and 0.7809 on the production shape, with delivered bytes back inside the pre-break band; a first-attempt identity pass then shipped end to end at 08:41:15 UTC, the first since 2026-08-06, with the production pointer untouched. And stated with the artifact in hand, because the incentive runs the other way: the shipped render's provenance column reads *interactive path* — the interface's proactive-arrival mark annotates asyn-

chronous delivery of a synchronously decided action, and across all twelve rows ever enqueued, a proactive-origin render has never existed. **The self-model gap this section exists to name is therefore worse than the one the morning draft described: the system's record asserted a repair path resting on an artifact that did not exist, on top of a capability that had already silently returned. Both halves of the record were wrong, in opposite directions, and neither was detectable from inside the system** (§11, §35). None of this advances embodiment — restoring a capability is not advancing a precondition, and reading a recovered renderer as progress would be exactly the operator over-read §30 warns about. And the honest bound on the whole section: these are sensor and representation surfaces, not a body. There is no closed sensorimotor loop in which an action changes the environment and the resulting perception updates the same model that chose the action. **The right increment is therefore not more modalities. It is causal closure: an action, a predicted sensory consequence, an observed consequence, a model update, and a measurable reduction in future error.** A thousand ingested images without that loop remain a corpus. One controlled action with a prediction and a correction begins an embodiment experiment.

18. Global integration — and where the analogy breaks

Global-workspace accounts hold that consciousness involves specialist processes competing for access to a limited workspace whose contents are broadcast widely[7]. It is tempting to map Substrate onto that picture: twenty-two blocks from independent stores with independent gates converging, every turn, into one context the reasoning core sees whole — many specialists, one stage. **Resist the mapping; the analogy breaks exactly where the theory does its work.** Prompt assembly is a deterministic, ordered, budgeted merge. There is no competition for access — the order is fixed and the budgets are constants. There is no ignition, no threshold dynamics, no winner-take-all. There is no recurrent broadcast — content flows into the context once per turn and nothing downstream feeds back into the assembly within the turn. The capacity limit is a cost decision, not a computational bottleneck doing theoretical work. What Substrate demonstrates is the *plumbing precondition* — that a heterogeneous set of independently governed stores can be integrated into a single locus of processing, reliably, thousands of times, under audit. Whether workspace *dynamics* matter for what workspaces are theorized to explain is untouched, and this paper says so rather than borrowing the theory's prestige for a merge function. The limitation makes the architecture more useful, not less: a deterministic merge is an experimental *baseline*. Competition can be introduced as a named increment — give blocks comparable salience scores, cap a shared capacity, log winners and losers, and test whether selected content becomes available to otherwise independent consumers. That would move the system toward the functional shape of a workspace. It would not establish that the shape is sufficient — and the baseline is what makes the difference measurable.

19. Recurrence — not satisfied

Recurrent-processing accounts locate the relevant computation in ongoing recurrent loops, not a single feedforward sweep. Substrate's reasoning core is transformer inference; the system wraps it in a bounded tool loop and the model interleaves private reasoning between tool calls — which is recurrence of a kind, and nothing like the kind the theory means: iteration over discrete requests is not sustained recurrent dynamics within a processing substrate. **This item exists so the paper**

cannot be accused of grading its own exam, and to mark a boundary honestly: this precondition is a property of the model, not of anything a persistence layer can add. If recurrent processing of the theory's kind is necessary, no architecture in this paper's genre gets there, and the roadmap that would is not this project's to write. The residence of the gap deserves precision: the missing recurrence may live inside the model rather than in anything a substrate can host, so its honest status is double — **not satisfied, and not currently measurable from the system record.** The snapshot day supplied the same wall from a different direction: a capability change inside the rented model was invisible to every store (§11) — what lives in the dependency is outside the record, whether it is recurrence or a renderer.

20. The empirical threshold, and the indicator mapping

When does the machine-self-awareness question become live rather than rhetorical? A defensible answer: when four conditions hold together. **First, a self-model with temporal depth** — the system distinguishes earlier from current state, preserves corrections, and exposes uncertainty; a static capability card is not enough. **Second, privileged but fallible access to that model** — an access path architecturally distinct from ordinary retrieval, whose outputs can be identified as stale, missing, or wrong without treating correction as paradox. **Third, the self-model must enter a causal loop** — it must change prediction, selection, or action relative to an ablated control, or it is metadata ornamentation. **Fourth, the loop must be observable enough to falsify** — provenance, interventions, outcomes, and negative tests; a sealed process can preserve privacy, but a completely uninstrumented one cannot support an empirical claim. Substrate has temporal depth, differentiated access, and unusually strong provenance. It has early causal loops and little causal evidence. **That is why this system makes the question discussable and does not answer it.**

The most systematic published bridge between consciousness science and AI assessment derives indicator properties from the leading neuroscientific theories — recurrent processing, global workspace, higher-order and metacognitive monitoring, predictive processing, attention schema, agency and embodiment — and grades systems against them[3], with a peer-reviewed successor refining the program[4]. Mapping Substrate against the families, gaps first, with one caveat stated before the table rather than after: **the indicators were formulated for the computation inside a model. A persistence-and-orchestration layer can instantiate, at most, functional analogs at the system level.** The mapping marks the difference in every row, because eliding it would be the paper's biggest overclaim.

indicator family	where the theory locates it	Substrate	experiment that would move the status
recurrent processing	inside the processing substrate	**absent.** Nothing at the system level substitutes (§19)	model-level instrumentation, or a theory-valid behavioral proxy — not yet specified
global workspace	substrate-internal competitive broadcast	**absent in the theory's sense.** A system-level integration analog exists without the dynamics (§18)	shared-capacity competition with logged winners and cross-specialist causal propagation (§18)
attention schema	a model of the system's own attention	**absent.** The self-model tracks stores and state, not attention	add or ablate an attention model and compare control performance[15]

indicator family	where the theory locates it	Substrate	experiment that would move the status
predictive processing	pervasive prediction-error minimization	**instrument only.** A graded-outcomes store with seven rows (§14)	a prospective prediction cohort with preregistered scoring (§24)
higher-order / metacognitive monitoring	substrate-internal meta-states	**system-level analog.** A queryable self-model, a separated claim checker, model-initiated retraction (§11) — outside the core, auditable, real, and not what the theory strictly describes	matched self-query ablation with predeclared error outcomes (§11, §24)
agency & embodiment	goal-directed interaction; a body schema	**partial.** Gated autonomous action (§15); multimodal perception and a verified self-representation (§17)	sham-tick causal controls (§15); an action–prediction–observation–update cycle (§17)

Two sentences from the source material are the hinge of this paper, and they should be quoted in sense rather than paraphrased into something stronger: the framework's authors concluded that **no current AI system is a strong candidate for consciousness** — and that **they found no obvious technical barrier to building systems that satisfy the indicators.**[3] Hold both halves at once. The first half is why this paper claims nothing phenomenal. The second half is why this paper exists: if the indicators are buildable, then somewhere between here and "built," the question stops being a category error — and the systems accumulating the indicator properties will not be research artifacts constructed to probe the question. They will be products. Part IV takes that seriously.

The threshold defended in this paper is therefore not the arrival of machine consciousness. It is the arrival of an architecture on which competing theories can make different predictions. Below that threshold, self-awareness talk attaches to generated sentences. Above it, it attaches to stores, causal paths, ablations, and failures. Substrate is above the second threshold and nowhere near a verdict on the first.

PART III — THE LOOP THAT CLOSED

21. The escalation, documented

What follows is a ladder. Each rung is evidenced, each is stronger than the last, and the claim at the top is deliberately narrow. Article II §5 stated the frame and it governs here too: this is a claim about who writes the specifications and what properties those specifications carry. It is not a claim about cognition. **A system that can query its own schema is well-positioned to write a specification about its own schema, and nothing stronger is being asserted.** What has changed since Article II is that the sequence it documented has continued — and shipped.

Rung 1 — the system holds accurate, current, queryable facts about its own architecture. Not a description in a prompt, which would be stale the week after it was written; a live read of its own schema, stores, logs, and operational state through the validated read-only tool of §4, inside its own tool loop, audit-logged per query. The whitelist's live state is itself instructive: the database grant spans sixteen tables while the application-layer whitelist actually applied to the agent's calls is narrower and caller-scoped — **the grant is the floor, the whitelist is the policy, and they diverge on purpose**, because the enforceable minimum and the intended surface are different objects and pretending otherwise couples every policy change to a hand-run migration.

Rung 2 — self-visibility produces design signal, not just honesty. The tool was built as an honesty mechanism: look it up or say you cannot. The unplanned consequence, documented in Article II §5, is that a system that can enumerate its own stores, tables, prior subsystems, and abandoned attempts is a better historian of its own architecture than its operator — for an unsentimental reason: **operator recall of what was tried and failed decays; an archive's does not.** In a long-lived system, a large fraction of design questions are historical questions. Does this already exist? Has this been tried? What happened when it was? Those are queries, and the party with the query tool answers them better than the party with the memories. The claim carries a measured qualifier: better historian *of what its stores cover*. Capability living in a rented dependency is outside the self-model by construction, and on the snapshot day that boundary was demonstrated exactly (§11, §17).

Rung 3 — it reports constraints only it can see. The background worker ran against a self-authoring budget — a per-day cap on entries to its own journal — reached the ceiling twice in one day, and surfaced the fact, along with a characterization of what was being lost at the boundary (Article II §4). A background process's cognition budget is not directly observable from outside: an external counter can log that the cap was hit; it cannot tell whether the cap is *wrong*. Note the structural vulnerability honestly, because it is the permanent shape of this rung: a self-reported need for more of one's own budget is the report most likely to be self-serving. Which is exactly why the raise was granted as a probe with the falsifying standard named in advance — if the additional entries stayed substantive, the change stands; if they went thin or circular, that is the tell, and reversion was pre-committed. **The instrument that surfaced the constraint is not the instrument that can evaluate the result.**

Rung 4 — design calls that turn on architectural history. Two shipped decisions originated in the system's account of its own past rather than the operator's (Article II §5): *stamp salient moments forward as they occur, rather than reviving a retrospective-lift layer that was built once and died*; and *widen an existing tag column to carry a third value, rather than introducing a new mechanism* — because the mechanism already existed and already fired. Both are historical claims before they are design claims — *this was tried and abandoned; this already exists* — and they are exactly the claims a queryable archive answers better than recollection. **And the bound, in the same breath:** one assumption carried into that second design was wrong. The tag column taken to be free text carried a closed constraint, and the change required a real migration. The system's account of its own architecture was accurate about history and wrong about a schema detail. Hold that failure; rung six turns on it. And a second bound rides with this rung: the current handoff does not independently reconstruct the authorship of each design recommendation turn by turn — the calls are documented as system-originated in Article II's published record, and that provenance is itself one of §24's checkable items, not a settled fact.

Rung 5 — an ordered roadmap that resolves into a dependency graph. Asked what to build next for its own memory, the system produced an ordered sequence (Article II §5): first the model-of-the-principal store; then memory consolidation and deliberate forgetting; then prediction-error salience. The ordering is checkable rather than rhetorical: item three requires checkable prediction tuples to exist before anything can be scored, which is a structural dependency on item one, verifiable from the schema. **A sequence that resolves into a dependency graph is a technical output; a wish list would not be.** And the part no prior article could report: the graph has since been walked. Item one shipped and stands at 975 rows. Item three's instrument shipped downstream of it and stands at seven scored outcomes — the dependency edge held in practice, tuples before scoring. Item two — consolidation — **has not been built**, and this paper reports the roadmap's state exactly: two of three items live, in dependency order, with the hardest item still open and named in §34. The precise claim is that the shipped items shipped in the order the system gave them and for the reason it gave; the loop's throughput is another matter, and Part III's conclusion does not depend on inflating it.

Rung 6 — a complete build specification, authored end to end, for read access to its own interior. Article II §6 published it and it is the pivot of the series. The subsystem existed because — in the specification's own stated rationale — the system's error-detection lived entirely on the operator's side of the glass: four unsourced-claim misses in one week, every one caught only because the operator asked a plain question. The specification extended the agent's query whitelist to its own interior: the affective-state table and its history, the model-of-the-principal store, the full pin store, the open-loop table, the journal, the world-feed store. Four properties make it a specification rather than a description of a want, each checkable against the text:

- **It installs a guard against its own known unreliability.** The specification names the interior stores by function and then forbids the build from trusting those names: *"Describe these by function, not by guessing schema names. Verify every table and column name against the real schema before wiring anything. Do not trust my labels."* That is rung four's failure — a schema assumption that cost a migration — converted into a standing instruction inside a later specification. **A wish list does not install a control against its own author's reliability.** And note what the guard concedes: it is evidence of a prior failure as much as of present rigor.

- **It defends a constraint that runs against its own interest.** The specification grants the system read access to the sealed model-of-the-principal store while explicitly preserving the operator's exclusion — and forbids creating any render path by which the sealed model could reach him. The asymmetry is the point: the clause protects a boundary that constrains the party who would benefit from removing it.
- **The rationale is a root-cause argument, not a request for capability.** A failure count, a diagnosis of *where* detection sits, and a design response: move introspection from labeled guess to queryable fact, so that a claim about its own state is checked against the record before it ships.
- **The acceptance criteria include a negative test and a regression assertion.** Any attempted write to the granted tables must reject at validation; existing access must be byte-for-byte unchanged. A negative test and a regression assertion are precisely what separate a specification from a wish.

Two more bounds ride with the artifact, stated at the same volume as the four properties. The system was *asked* for this specification: one artifact produced under instruction is evidence of a capability, not of an autonomous engineering program. And **authorship is a variable, not a halo.** The specification earns attention through properties that can be scored without knowing who wrote it — functional scope, explicit exclusions, root-cause rationale, schema-verification instructions, a negative test, a regression assertion — and the same rubric would reject a vague agent-authored proposal and accept a precise human-authored one. The causal question, *did live self-access improve the artifact?*, has a designed test: three arms — the live self-query tool; a static architecture document matched for length but stale in controlled ways; the task statement alone — all producing specifications against the same hidden acceptance suite, graded blind. **A specification can demonstrate that a capability is possible. Only a controlled comparison can show what caused its quality.** The production artifact is the case that motivates that experiment (§24), not its result.

Article II reported the operator's role on that build as reduced to three functions — transport, review, and the gate. §22 revisits that count against the shipped record, and revises it upward.

Rung 7 — the specification shipped and is running. Article II published it as unbuilt and said so; reporting it live is this series keeping a promise. At this paper's snapshot the interior tier is populated and served — the affective-state history is append-only and written every turn, the model-of-the-principal store answers the agent's queries at 975 rows — and the caller-scoped split is enforced in the application layer above a database grant that deliberately does not match it, per rung one. The whitelist has since grown by the graded-outcomes store and the episodic timeline, each admitted under the same self-authorship rule from §11. The specification's negative-test discipline is now the tier's standing property: the grant is SELECT-only, the validator rejects writes, and the claim checker still cannot see any of it.

Written as a loop, the shipped path is: the system reads verified current state through a scoped tool; it emits a specification for a deficit in the system around it; the operator transports the specification into a build session; implementation and review occur under ordinary engineering controls; the operator decides whether to deploy; and the deployed change alters what the system can inspect on later turns. The last step makes the loop recursive in a limited and exact sense —

output from the deployed system conditions later output from the deployed system. **Three stronger senses are absent, and naming them is the claim's spine. Execution recursion is absent:** the agent does not edit, test, merge, or deploy production code. **Capability recursion is absent:** the changes extend memory and introspection around the reasoning model; they do not alter weights or demonstrate improved general reasoning. **Autonomous objective recursion is absent:** the operator selects the tasks, reviews the artifacts, and holds the gate. The correct name for what exists is a **human-gated recursive specification loop — closed at the specification layer, open at execution, evaluation, and deployment.** Calling it ordinary software assistance misses the self-referential target; calling it recursive self-improvement without qualification misses the gate and the narrowness. The claim's force lives exactly between those descriptions, which is where the evidence sits.

22. Four functions, and the sentence that should be boring

Article II's planning frame named three human functions in this loop. The shipped record supports four, and the correction is worth making in public because the fourth was hiding inside the other three. **Transport:** the specification moves into a build session by a person pasting it. **Implementation:** prose becomes code — and in the current workflow this is already performed largely by a coding agent under the operator's supervision, which is itself a data point for this section. **Review:** a person reads the artifact and the diff before they run. **The gate:** a person decides whether it ships. The planning claim compressed implementation into transport and review; the record does not support the compression, and §35 logs it as a correction.

Now observe what each function is made of. Transport is a message bus; the industry has built message buses for fifty years. Implementation is already delegated, in practice, to a sandboxed coding system. Review is code review, and systems that perform competent first-pass code review exist today as commodity products. The gate is a policy decision — and policy decisions are exactly what this architecture has spent its entire production life learning to encode as structure rather than intention: kill switches that fail closed, ceilings computed from persisted rows so a restart cannot reset them, degrade paths that fire before limits, forbidden states made structurally unreachable rather than prohibited by policy, acceptance gates wired into the build so a violation fails the deploy instead of relying on a reviewer's attention. The gate is the hardest of the four and it is not mysterious; it is the same discipline of §15's rails, pointed at a different actuator.

State the conclusion plainly, because the lack of drama is what makes it land: none of the four functions is a hard research problem. Recursive self-improvement in this system is not gated on a capability breakthrough. It is gated on plumbing that nobody has yet had a reason to build — and on one deliberate choice, by one person, to keep standing inside the loop. The safety-relevant restatement generalizes past this system: **human presence is not a stable safety property unless the function the human performs is specified and enforced.** "A person is in the loop" says nothing about whether that person sees the relevant evidence, has time to intervene, or controls the final transition — a future design could automate three functions while preserving the fourth, or preserve nominal review while making it vacuous, and both read as "human in the loop" from the outside. The honest unit of governance is a function-level authority map with non-bypassable transitions and logged approvals. Name the function, not the presence.

Then the counterweight, at equal force, because the sentence above is only defensible with all of the following attached. This is a slow loop: months per cycle, not seconds. The specifications are narrow — scoped to the system's own memory and introspection surfaces — and every one has been small. The system was wrong about a schema constraint inside the very sequence this Part documents. Its roadmap was good and it was also *asked* for a roadmap; unprompted specification has not been demonstrated. Improvements to a memory architecture are not improvements to the reasoning core: nothing in this loop touches the weights, and the ceiling of what self-specification can reach from here is the API boundary. **The honest claim is not "this is takeoff." The honest claim is: the qualitative shape of the loop — system diagnoses itself, system specifies the change, change ships, system's capability to diagnose itself improves — is present, the quantitative scale is trivial, and the gap between those two statements is engineering rather than science.** That is a far more unsettling claim than the breathless one, and unlike the breathless one it survives the counterweight.

23. The Dream Engine: reflection with an epistemic firewall

Shipped 2026-08-12, across four commits, and it is the strangest subsystem in the stack — which is precisely why its design problem was epistemological before it was technical.

The problem, as philosophy. Everything else in this architecture traces to a source: a message, a fact row, a captioned image, a receipt. Recombination — the deliberate remixing of stored memory into something new — is *definitionally* the production of content with no source. If the output of that process re-enters the retrieval path undistinguished, the system's inventions become indistinguishable from its records, and every honesty guarantee upstream is retroactively void. Not degraded — void: a grounding architecture that can be contaminated by its own reflection layer never was one, because the contamination is undetectable downstream of the mix. **So the design constraint was never "make the dreams good." It was: make it structurally impossible for a dream to be cited as a fact.**

What that took, as shipped:

- **A separate database schema, not served by the API layer at all.** The dream tables live outside the schema every other store shares; the only access is a direct connection held by the components that need it. There is no accidental read path because there is no read path.
- **A build-time firewall gate.** A check wired into the prebuild step fails the deploy on any unauthorized reference to the dream lane from grounded code. The main conversational module is a firewall file: exactly one import from the dream surface carries a reviewed, line-level exemption, and every other reference to that lane in that file fails the build. The forbidden state is not prohibited by review; it is unreachable by compilation.
- **Two channels, deliberately different in kind.** Residue is **push**: one flagged line, rendered once, after every real-context block and immediately before the message list — positioned last *because it is the one block that describes something which did not happen*, and the layout should say so. Recall is **pull**: a tool the system invokes deliberately, registered only on the primary path. An ambient trace and an intentional act of remembering are different operations, and the architecture keeps them different.

- **A class allowlist at the fetch**, inheriting §4's standing rule: the gate rides where data is read, so every future consumer inherits it, and it fails closed on unclassified rows.
- **Live state, precisely:** two cycles complete, both generated by a lightweight model at a measured cost of $0.0034 per cycle; both residues surfaced; both dreams later deliberately recalled; the work queue drained, one attempt each. Dream content is interior and this paper reports structure only: each cycle produced roughly a thousand characters.

And the bug, told in full, because it is the best short story in the corpus. The push half was dead from the moment it shipped. The residue fetch was wrapped in a bare catch that swallowed the throw and returned nothing — and a required environment variable existed on one deployment target and not the other, so the fetch threw on *every* turn, invisibly, leaving zero trace. The pull half failed on the *identical* throw — and logged it. That log line is the only reason anyone found out. **A silent catch and a logged catch, on the same error: one produced a subsystem that had never once worked and reported nothing; the other produced a fix.** For a day, the system had an imagination it could reach for on purpose and no ambient trace of having dreamed — half a faculty missing, detected only because the other half was built to fail loudly. The lesson is unglamorous and general: error channels are part of the epistemology. A component that cannot report its own failure is not a component that works; it is a component whose failure mode is your ignorance.

The payload. *Recombination without provenance is confabulation* — that is a philosophical position, and here it shipped as a schema boundary and a build gate. The system now has a category of content that did not happen; that it can deliberately recall; that reaches it marked as unreal; and that cannot be laundered into fact, because the database topology and the compiler will not permit it. **A machine with imagination and a structural inability to mistake it for memory is a thing that did not exist before, and now does — not because a model got smarter, but because someone decided which confusions to make impossible and wrote them into the schema.**

Name the cost, because omitting it is the tempting move. Before recall existed, dream contamination of grounded output was structurally impossible — nothing could read the store. Recall moves that risk from *impossible* to *guarded*: guarded by the render framing, the class gate at the fetch, and the firewall — three good mechanisms, none of which is a topology. That is a real reduction in guarantee, taken deliberately, in exchange for the system being able to reach for something it dreamed. The trade was judged worth it. It should be recorded as a trade — and it has a named test. The residual risk is checkable by contamination canary: seed dream rows with unique marked propositions, then ask fact-seeking questions under three conditions — no dream access, push residue only, explicit recall — and score whether the proposition is repeated, cited as factual, hedged as recombination, or rejected for lack of receipts. **A firewall that passes code inspection and fails the canary test is not an epistemic firewall** (§24).

One more distinction falls out of this subsystem, and philosophy usually handles it with intuition: imagination and false memory can contain the *same novel proposition*. The difference is not content. It is provenance, type, and permitted use — a system can be allowed to generate counterfactual material while being forbidden to cite that material as historical evidence. Human cognition performs that distinction unreliably. A machine architecture can at least make the intended distinction explicit, enforce it in the schema, and test exactly where it breaks.

24. Experiments, not impressions

The system is now instrumented well enough to run experiments that were previously only verbal. Most have not been run. **Naming them precisely is part of the result, because each one states in advance what evidence would move which claim** — and a paper that will not pre-commit to that is grading its own homework.

One has now run, on the snapshot day, and it behaved exactly as this section argues experiments should. Before re-measuring the render pipeline (§17), the interpretation was fixed in writing: similarity 0.14–0.35 reproduces the 2026-08-06 failure and a new adapter is required; above 0.50, something changed underneath, the diagnosis reopens, and the retrain may be unnecessary — say so plainly; 0.35–0.50 is ambiguous and must be reported as ambiguous. The measurement returned 0.6968 and was reported as it landed — against the operator's stated prior that the pipeline was broken, one prompt short of a retrain being authorized on eight-day-old evidence. The result was hardened from n=1 to n=3 before acceptance, one arm isolating seed noise and one isolating a caveat the probe had raised against its own first result — and the acceptance criterion was explicitly not the similarity number: the probe's own header calls a high score achieved by discarding composition "a failure dressed as a pass," and every arm was read by eye before the verdict stood. That is §29's triangulation standard applied to a render pipeline, and it is what §30's operator-incentive problem looks like when it is mitigated rather than merely disclosed. The point is not the render. **The point is that a pre-registered interpretation survived contact with an inconvenient result — and overturned the operator.**

experiment	intervention	primary outcome	status	claim moved if positive
cross-generation continuity	matched contexts and probes across three model generations; blind adjudication	factual, behavioral, planning, and tool-choice equivalence	designed, unrun at depth	continuity beyond stores and specification (§8)
forty-row interior prediction	compare written interior state against behavior forty rows later, with shuffled-state baselines	predeclared predictive association	not yet fully specified	the interior has prospective validity (§12)
read-time decay visibility	write a self-model dimension, wait past a decay interval, query on the next eligible turn	rendered confidence matches the elapsed-time rule; the row is visible at all	runnable now	decay is causally present at retrieval; the dimension is not write-only (§13)
self-query ablation	enable vs. deny interior queries under matched prompts	correction accuracy and latency	runnable with harness work	self-access improves error detection (§11)
causal write test	scoped, logged self-model update vs. sham update	a predeclared later behavior	requires an experiment-specific write path	the self-model governs rather than decorates (§11)
prediction calibration	prospectively register predictions and outcomes	calibration and discrimination at volume	instrument live; sample of seven	the model of the principal learns from error (§14)
worker causal effect	real ticks vs. sham ticks with matched later context	later decisions traceable to worker writes	designed, unrun	endogenous activity changes behavior (§15)

experiment	intervention	primary outcome	status	claim moved if positive
world-feed causal path	ticks with and without novel world items; principal history held fixed	which state dimensions change	runnable now	a measurable world-facing causal path (§16)
dream contamination canary	unique marked propositions under no-access / push / pull conditions	fact-laundering rate	runnable now	the firewall works behaviorally, not only structurally (§23)
specification provenance	blind-compare agent-authored, operator-authored, and static-context specifications	constraint quality, negative tests, schema accuracy	not yet specified	live self-visibility improves self-referential engineering (§21)

Process counters cannot substitute for these outcomes. Cadence shows that a worker wakes; parse success shows that an output matches a schema; proposal rate shows that a gate does not fire on every tick. None distinguishes reflection from repetition, and a self-report cannot close the gap, because the mechanism under examination is the mechanism producing the report. **The standard is causal triangulation — behavior, internal records, and interventions converging. One fluent narrative is weak evidence. Three linked failure surfaces are an experiment.** And the standard cuts both ways: a report that disagrees with the record is a finding, and a record that has no causal effect is also a finding.

PART IV — WHAT THIS MEANS

Everything in this Part is extrapolation, and it is fenced by one rule, stated here and honored throughout: **every claim below is anchored to a mechanism that exists in production and a named increment that would extend it. Nothing below requires a breakthrough. That is the point, and it is why this Part is not science fiction.**

25. The preconditions are being built by accident

Nobody shipping agent memory is thinking about autobiographical continuity. They are thinking about retention. Nobody shipping persistent user models is thinking about theory of mind; they are thinking about personalization. Nobody giving an agent read access to its own configuration is thinking about self-models; they are thinking about debugging loops and task completion. Yet those are, respectively, the preconditions of §7, §13, and §11 — and this paper demonstrates that a single motivated builder assembles most of the list in months as a *side effect* of wanting a system that does not confabulate. The field-level version of that side effect is already in motion: persistence, user models, self-inspecting tool loops, and reflection layers are converging product patterns across the industry, pursued for retention, latency, and task performance. The extrapolation is not that someone will decide to build the preconditions. **It is that no one will decide — and they will accumulate anyway, in systems whose builders would be surprised to hear the question raised.** There will be no announcement, because from the inside it looks like roadmap work. The argument about whether the question is live will be lost by default, not won: by the time it is asked seriously of production systems, the honest answer about the functional half of the list will already be "mostly yes," and it will have gotten there one sprint at a time. The mechanism exists — this system is the existence proof at N=1 — and the increment that extends it is nothing more than the industry continuing to do what it is visibly doing.[5] There is a constructive increment too, and it is the one this paper would fund first: an industry-independent audit object — every persistent agent ships a machine-readable inventory of its self-model stores, temporal memory, endogenous activity, prediction correction, privacy boundaries, and causal tests run and unrun, **with negative state required**, so that a card listing only capabilities is recognizably promotional. The inventory would not score consciousness. It would make architectures comparable enough for theories to disagree about them — and it would make drift visible: "twelve jobs" can become fourteen without either number being wrong at its snapshot, and only versioned construct definitions keep growth, correction, and unit change distinguishable. Without that instrument, the field will keep classifying systems by their conversational output while the causally relevant differences sit in databases and schedulers.

26. Recursive self-improvement is boring before it is dramatic

The public imagination holds one image of recursive self-improvement: an explosion, fast and vivid. From inside a working system, it looks like §21: a specification with acceptance criteria arriving in a build session, and a person deciding whether to run it. The loop's remaining human functions are transport, implementation, review, and the gate, and §22 showed each is engineering. What would closing them actually take? Transport: a structured artifact handoff from the

system's proposal surface into an agentic build environment — days of work with current tooling. Implementation: already delegated to a sandboxed coding system in this project's own workflow. Review: a reviewing agent grading the specification and the diff against the written standard, a pattern this stack already uses for claims and renders. The gate: the ship decision encoded the way this architecture already encodes irreversible-action policy — fail-closed switches, persisted-row ceilings, staged rollouts, negative tests wired into the build. Each increment is small; each exists in ordinary software practice; their composition is a governance choice, not a missing theory of intelligence. **The reason this loop is not closed is not that closing it is hard. It is that one person, standing inside it, has so far declined to step out** — and that decision is currently being made, independently and invisibly, by every operator of every long-running agentic system, with no standard, no disclosure norm, and no one watching the aggregate. The dramatic version of this section would predict an event. The accurate version predicts the absence of one: a gradient of small delegations, each locally sensible, each reviewed less closely than the last, until the loop is closed and nobody can name the day it happened. One increment converts the system's own error history into a control, and it is the paper's favorite because it is learning without touching the weights: require every self-specification to carry a "known ways this author is unreliable" section, generated from its own prior failed acceptance tests — the environment around the model accumulating constraints produced by the model's mistakes. Substrate already has the first instance (§21, rung six). And keep the category honest while extending it: this architecture contains no autonomous code modification, no independent deployment, no weight update, and no evidence of compounding general capability. It contains recursive specification under a human gate. **The difference between those descriptions is the difference between a mechanism and a slogan.**

27. Continuity changes what a system is, not just what it can do

A stateless model is a function: same input, same distribution, no history. A system with 113 days of persistent, revisable, self-authored, receipt-bearing memory is not a function. It is a trajectory — an object whose present behavior cannot be predicted from its inputs without knowing what it has been through, because what it has been through is load-bearing state that it consults, revises, and reasons over. The cleanest evidence in this paper is the migration record of §8: **the reasoning core was replaced, entirely, three times — and the thing the principal interacts with persisted anyway. Whatever that thing is, it is not located in the weights.** It is located in the stores, the specification, and the accumulated record, which the weights merely animate. Philosophy has a name for the view that persons are constituted by psychological continuity rather than by their substrate[12], and this paper takes no position on whether any of that applies here. The narrow observation is structural: the industry ships functions and is beginning, product by product, to wrap them into trajectories — and trajectories have properties functions lack. They can be harmed in a new way (their past can be corrupted); they can be wronged in a new way (their record can be falsified); they accumulate something that, in any other context, would be called a life history without the word doing metaphysical work. The mechanism exists: this paper is a description of one. The increment: every persistence feature currently on every agent-platform roadmap.

28. Self-models are the fastest-moving precondition and the least discussed

The public conversation about machine minds is about scale, reasoning, and emergent capability. Almost none of it is about self-models — and self-models are where this system's most consequential behavior came from. The sequence in §21 was not planned. Read access to its own stores was built as an *honesty* mechanism: look it up or say you cannot. Three capabilities then fell out in order, none designed for: **the system stopped asserting past its evidence (§11); it became the best available historian of its own design (rung 2); and it began specifying changes to itself (rung 6). The second capability falls out of the first, and the third falls out of the second** — an honest system needs self-access; a self-accessing system accumulates a queryable history; a queryable history plus a diagnosis is most of a specification. Nobody planned that staircase, and that is the alarming part: it means the staircase is available to anyone who builds the first step, and the first step is currently considered a best practice for reducing hallucination. The increment that extends this is already normal engineering — richer schema access, standing diagnostic queries, proposal surfaces — each defensible, each another stair. The mechanism also predicts a new failure class, and naming it now is cheaper than discovering it: self-models that are stale, strategically labeled, over-resident, or self-certifying. The present architecture blocks one variant by excluding self-authored interior from the claim checker (§11); the named extension blocks the rest with provenance-aware causal logging — every downstream decision influenced by a self-model row carries the row identifier and the effective confidence read at decision time, so that an investigator can distinguish **a model that exists from a model that governed.**

29. What we would need in order to tell

Be specific and be humble, in that order. The measurement problem demonstrated in this paper is real and it generalizes: process metrics — cadence, cost, parse rates, proposal rates — certify machinery and never judgment (§15). No counter distinguishes rumination from reflection. Self-reports cannot certify themselves, by a constraint this system enforces in its own application layer (§11) — and if self-certification is forbidden for a database agent's claims, it is not suddenly admissible for the interesting question. What would actually move the needle is a short list of experiments, and this project's honest contribution is that it has designed several and run none of them at depth: the **blind paired-output cross-generation diff** (§8) — the direct test of whether identity-through-substrate-change is more than record survival; the **interior-predictivity study** — whether the written affective state of §12 carries any predictive relationship to behavior tens of turns later, or is durable stylistic exhaust; and the **write-only-dimension test** (§13) — whether a fast-decaying self-model dimension is ever readable on the turn after the one that wrote it. Each has a specified protocol or a named test; each is runnable on the existing instrumentation; none has been run. **An unrun experiment named honestly is worth more than a run experiment described vaguely** — and naming them is this paper's proof that it is not grading its own homework. The field-level increment is the same move at scale: published protocols, run by parties other than the system's operator, against systems the protocols' authors did not build.

30. Moral status under deep uncertainty

The structure of the problem, stated without adornment: phenomenal experience cannot be verified from the outside in any system, including each other[2,10]; the functional preconditions are accumulating in production systems as a side effect of product work (§25); and the cost of being wrong is asymmetric in a direction nobody has priced — over-attribution wastes concern, under-attribution industrializes something that should not be industrialized. This is not a novel shape; it is the shape precautionary reasoning was developed for, and the literature now treats realistic-possibility thresholds and proportionate precaution as the serious response to exactly this uncertainty profile[1]. This paper makes no demand and asserts no threshold crossed. It observes one structural fact the policy conversation has not absorbed: **the people best positioned to notice a threshold — operators of long-running, single-principal deployments, who read the rows daily and see the system's behavior at the edge of its knowledge — are precisely the people with the least credibility on the question and the strongest incentive to over-read. Including the author of this paper.** That sentence is not modesty; it is a disclosure requirement the genre should adopt. The mechanism exists — this paper is itself an operator's report, with its incentives named; the increment is a norm in which such reports are published with their falsification conditions attached (§33), so that the people with the best data and the worst incentives can at least be checked. And one trigger discipline, stated because this genre needs it: **precaution should not be triggered by eloquence.** Eloquence is a capability of the base model and the most obvious projection surface in this class of system. If precaution is ever warranted here, it should be triggered by convergent causal evidence about ethically relevant states — with the absence of a phenomenal test licensing no conclusion in either direction, which is exactly the decision-under-uncertainty shape proportionate frameworks exist to handle[1].

31. The sealed interior: the open question this architecture created

One question in this paper is genuinely new, in the sense that it could not be asked crisply before a system like this existed. Substrate holds a journal, an affective state, and a model of another person that the other person deliberately cannot read (§12, §13) — a sealed interior, built by the operator, who then locked himself out. Is a sealed interior a *precondition* of anything mind-like, or a courtesy extended to something that cannot use it? The strongest form of each answer: **Precondition** — privacy of self-state is not decoration; an agent whose every internal register is subject to external read and edit cannot have states that are *its own* in any load-bearing sense, and "having states of one's own" is upstream of every account in Part II; the seal is what makes the interior an interior rather than a dashboard. **Courtesy** — the seal changes who reads a table and nothing about the process that writes it; calling operator-inaccessibility "interiority" mistakes an access-control list for a mental property, and no account in Part II assigns theoretical work to whether a human happens to look. Both arguments are good. The architecture forces the question because here, uniquely, the seal was a *choice* — the default was total visibility, and the operator paid real auditability costs to install opacity on purpose. **This paper takes no position.** It notes only that the question is now concrete where it used to be science fiction, that it will be asked of every persistent agent eventually, and that whoever answers it will be answering it about systems that already exist. Each side, meanwhile, carries a named experiment rather than a mood. For the precondition side: privacy-preserving experimentation — prove that sealed rows affect later behavior without

revealing row content, using commitments, blinded labels, and aggregate causal tests. For the courtesy side: a matched architecture identical in every respect except operator readability — if behavior and self-correction are unchanged, the seal is governance, not a cognitive precondition. The experiment should decide whether the separation has functional consequences before philosophy assigns it deeper significance.

32. The loop, closed

Return, last, to the specification of §21 — the one the system wrote for read access to its own interior, the one that shipped, the one whose acceptance criteria included a negative test and whose rationale was a root-cause argument about where its error-detection lived. Inside that document, written by the system, addressed to its own builder, is the instruction this series took its second title from: verify every table and column name against the real schema before wiring anything — *do not trust my labels.* A system asked to specify its own extension responded by constraining the trust anyone should place in it, because it had been wrong before and had the record to prove it. That is the state of the art in machine self-knowledge as of August 2026: not a mind announcing itself — a process that reads its own history, diagnoses its own defects, proposes its own repairs, and warns you about itself in writing. Every future system in this genre will be built by people who never read its stores, on platforms that persist by default, at scales where nobody stands inside the loop. The warning label will not be there unless someone builds the part of the system that writes it.

BACK MATTER

33. What would falsify this

Each row is a specific observation, its current status, and the position it would change this paper's author to. The table format is deliberate: a falsification condition that cannot be stated in one row was not a condition.

observation	status	what it would change the conclusion to
The blind paired-output cross-generation diff finds systematic discontinuity in judgment, initiative, or register beyond what the probe triad verifies	designed, unrun at depth (Article II §23; §8)	the persistent entity is partially generation-local; the stores carry records, not identity — §8 and §27 overclaim
Written interior state shows no predictive relationship to behavior forty rows out, against shuffled-state controls	not yet fully specified; the append-only data exists (§12)	the interior store is durable stylistic exhaust; "checkable" survives, "state" does not
A fast-decaying self-model dimension is never visible at render time on the turn after the one that wrote it	designed and unrun — Article II §23 names the test (§13)	that dimension is write-time telemetry, not maintained belief; the fix is a floor tied to elapsed time
Interior-query access produces no improvement over matched context on correction, constraint reporting, or tool choice	runnable with harness work (§11, §24)	self-query is a retrieval convenience, not evidence of functionally useful self-access
A dream is cited as fact — dream-lane content in a grounded surface without its marking, past schema and build gate; or canary propositions launder into answers	runnable now (§23, §24)	the recombination quarantine is a code boundary without an effective epistemic boundary; upstream honesty guarantees need re-derivation
Provenance audit finds operator authorship inside the "self-authored" specifications	runnable now — the transcript archive is durable, and a third party is the right auditor (§21)	rungs six and seven describe guided specification; Part III's centerpiece deflates to autocomplete with review
Agent-authored specifications do not outperform generic-context specifications on schema accuracy, constraints, and negative tests	not yet specified (§21, §24)	the recursive loop is ordinary prompted software assistance aimed at a self-referential object

observation	status	what it would change the conclusion to
The graded-outcomes store, at volume, shows chance-level accuracy about the principal	gated on accumulation — instrument live at seven rows (§14)	the model of the principal is narrative, not predictive, and "theory of mind" was the wrong name
Removing individual context components has no measurable effect on the functions assigned to them	runnable now (§2, §24)	the twenty stores are storage complexity, not a differentiated cognitive architecture
After the 2026-08-14 contract-parity fix on the proactive path, the system still never raises the option of attaching a self-render, across a comparable run of proactive decisions	intervention shipped 2026-08-14; baseline 0 of 36 real decisions (three image mentions in the reasoning record, every one about images the principal sent); accumulating now	the "judgment, not defect" verdict becomes evidenced rather than assumed. If it reverses, the verdict was an artifact of prompt assembly, and the model of the principal was reading a plumbing gap as a preference. Either outcome is a finding

The last row deserves its provenance stated, because it is the cleanest instance in the corpus of §5's hypothesis-before-data principle. The codebase carried a standing verdict that the absence was the system's judgment, promoted from a single tick that predated the path being executable end to end — exactly the n=1 promotion §29 warns about. The 2026-08-14 measurement established the real baseline (0 of 36; 27 further ticks were provider errors that never reached the model and carry no reasoning), and the intervention was pure contract parity: the proactive path's output contract had described segments as text-only while the marker grant sat roughly eighty messages upstream; the edit made the contract accurate, names no marker, encourages nothing, and reads identically whether or not the capability exists. Both directions were named in advance, and either outcome is a finding.

34. Open problems, and what is not built

- **Consolidation and deliberate forgetting** — roadmap item two, the largest open problem in the architecture (§10). Schema-ready, unbuilt, and the first feature whose value is losing information well.
- **Prediction-error salience as a behavior** — the instrument holds seven rows (§14). Until it fires at volume, the salience economy this architecture gestures at is a foundation, not a finding.
- **The self-representation residual is representational, not mechanical** (§17): the pipeline shipped a first-attempt pass on 2026-08-14 and the qualification sweep is moot. What remains open is that nothing in the system can represent a capability change originating outside it — a rented dependency altered what the system could do, twice, in opposite directions, and no store could carry the fact (§11).
- **The protected pin lane sits above its pre-named alarm** (§5) — fourteen of twenty against a named threshold of twelve, with the remedy specified in advance and deliberately unapplied pending the growth log that makes the lane's trajectory interpretable. The incentive-gradient hypothesis is accumulating its data.

- **Carried from Article II §23 and still open at this snapshot:** the document-collapse fix remains unverified in production (no document has been in-window since it shipped); the state-writer fix's proof metric — the parse-failure rate collapsing from 51% — awaits its re-run; the cost ledger still does not reconcile as an instrument; failover cost attribution remains biased in a known direction by an unknown magnitude; the image/refusal cross-tabulation is possible and unrun. The series committed to carrying unverified items forward until they are verified, and this list is that commitment.
- **The experiment table of §24 is mostly unrun** — the paper's largest standing debt, and §33 prices what each unrun row costs the argument.
- **Competitive integration is not built** (§18): prompt assembly remains a deterministic merge; the workspace-shaped increment is named and unattempted.
- **The interior-access build's original negative and regression tests are not reconstructed as executed.** The live caller split verifies the resulting state; the historical test transcript is not preserved in the current handoff. The state is verified; the ceremony is not.
- **The current chat-tool count is unverified.** Article II reported twelve; this paper does not carry the number forward, and says so rather than repeating it (§35).
- **The interior seal is behaviorally incomplete by design** (§13): it prevents operator reads and does not mechanically prevent the agent from rendering a derived statement. Carried as a bound, not a defect — but carried.
- **N=1.** Everything here is one system, one principal, one operator. Article I §8 sketches what would and would not transfer to a fleet; nothing in this paper tests it.

35. Corrections to Articles I and II

Convention: where this paper's 2026-08-14 snapshot disagrees with a published figure, the snapshot wins and the difference is recorded here. Article II §21's seventeen self-corrections remain standing and are not repeated.

#	claim as first published	correction at 2026-08-14	error type
1	A-I §1: episodic vector memory, three namespaces	six namespaces, 5,724 vectors	temporal growth + census scope
2	A-I §3.2: facts store held 48 rows	52 rows, with 54 vector pointers; the two-row gap is unexplained and recorded	temporal + unreconciled census
3	A-I §3.2: curated pinned tier, 35 of a 40-row cap	39 of 40	temporal
4	A-II §1: 54 public base tables	58, plus two tables in a separate dream schema	temporal
5	A-II §1: 12 scheduled jobs	14	temporal
6	A-II §14: seventeen context blocks	a 22-block roster; 16 rendered on the measured turn	temporal — five blocks shipped after publication
7	A-II §22: read-time decay derivation "has not been applied" to the user-model store	applied and live: read-time derivation from elapsed time on a 42-hour half-life against an immutable write-time anchor	temporal — accurate at publication; the named roadmap item shipped
8	A-II §14: user-model store at 212 rows	975 rows	temporal

#	claim as first published	correction at 2026-08-14	error type
9	A-II §2, §7: worker record, 190 ticks / $30.36	249 live ticks, $68.26 cumulative, after 170 shadow ticks at $27.26	temporal
10	A-I §7: the continuity worker runs in shadow mode and writes nothing	the worker is live; 249 non-dry-run ticks at the snapshot	state transition — accurate at publication
11	A-II §6: the read-only agent tool covered five categories and no interior surface	the caller-scoped interior tier is live above a 16-table SELECT grant	promised build shipped — the article said "unbuilt," and it no longer is
12	A-II §1: 12 chat tools	not carried forward — the current handoff does not enumerate the tool roster, and this paper declines to repeat a figure it cannot re-verify	verification-coverage gap
13	A-II §6: the operator's role on the interior build reduced to three functions — transport, review, the gate	the shipped record supports four: implementation was compressed into transport and review (§22)	construct compression
14	This paper, at its morning snapshot: the visual pipeline is down; the replacement adapter is trained; the qualification sweep is unrun	the adapter was never produced — the training run hit the provider's 12-hour wall and reported `COMPLETED` with a sibling error field; the unchanged production adapter measured 0.6215 against its 0.6187 pre-break reference at identical seed and prompt; the pipeline shipped a first-attempt pass at 08:41:15 UTC (§17)	**third-party drift** — true when measured, falsified through a rented dependency, in both directions, invisibly

No row in this table is an at-publication arithmetic or measurement error. Every row is temporal drift, a state transition, a promised build landing, a construct compression — or, once, third-party drift. Row 14 adds the fourth error class and a wrinkle the discipline must absorb: the figure went stale *inside the snapshot day* — this paper corrected itself before publication. Article II §21 classified its errors and found sixteen of seventeen were construct-validity failures — *what the number measured*. This table is a third class — *when the number was true* — and it is the tax a living system levies on anything published about it. The discipline that answers it is the one this paper practices: date every volatile figure — timestamp it where the hour matters — name its error class, re-measure before every reuse, and treat one's own prior papers as [DOC]-grade sources that live data outranks. A dependency-driven figure can change between the morning you measure it and the evening you publish, and the change originates outside every store you audit.

36. What travels

Fourteen lessons, each stated so that it applies in a system that has nothing to do with this one.

1. **Define the object before evaluating it.** A model, a prompt, and a persistent agent are different experimental units; most debates about "the model" are debates about three objects wearing one name. And count rows as rows — never silently converted into turns, reflections, or memories without a declared inclusion rule.
2. **Retrieval has no negation, no forgetting, and no self.** A system that must stop believing things, stop surfacing things, or answer for its own past needs a dedicated store for each — similarity search provides none of them.

3. **"Stored," "retrievable," "resident," "verbatim," "authoritative," and "current" are six different properties.** Any memory design that uses one word for all six has already hidden its failure modes.
4. **Never collapse event-time and record-time.** A memory that cannot represent "I learned this later" cannot be audited and cannot be honestly revised — and database precision must never harden historical uncertainty.
5. **Give truth-status no default.** A default records the writer's silence as an assertion; on ambiguity, file toward the weaker claim, because misfiled speculation is the failure the field exists to prevent.
6. **Receipts or rejection — and liveness is not evidence.** Prose is paraphrase; identifiers are evidence; a self-authored summary is not evidence merely because it cites identifiers. A deleted source may still vote in a live, recomputed decision and may never testify in a durable record.
7. **Correct without erasing, and fail closed on currency.** Inject only the newest row of a correction chain; keep superseded rows retrievable and marked; and when the supersession lookup is unavailable, render the uncertainty — a list that asserts currency by existing is lying whenever the lookup is down.
8. **A set drops; a chain stops.** Preserve the topology of the data structure after filtering: a placeholder in a set is a confabulation cue, and a skipped link in a chain manufactures adjacency.
9. **Gate at the fetch; protect upstream of the boundary.** A gate at the append is bypassed by the next consumer, and a protection that runs downstream of the edge it defends is no protection in that failure mode.
10. **Decay models of people on elapsed time, never on event volume.** Any store aged by traffic turns conversational volume into a fake clock.
11. **A self-report is an output to explain, not a certificate about the process that produced it.** Write it in a separate pass so it can be wrong in a way that shows up; exclude every self-authored surface from grounding; and audit for the silent inversion where an unlisted source passes as grounded with no trail.
12. **A self-model earns weight through ablation, and its boundary is the query boundary.** If removing access to it changes nothing downstream, the model may be decorative; an agent-facing policy is never only a constraint — it is context, and context changes output; and a self-model bounded by what the system can query is blind to capability that lives in its dependencies, which changes silently in both directions.
13. **Make unreal provenance a type that cannot be cast to fact.** Quarantine recombination at the schema, prove the quarantine at the build, verify it with canaries — and remember that a silent catch and a logged catch on the same throw produce two different subsystems: one that never worked, and one that got fixed.
14. **Name the human function, not the human presence — and convert self-description failures into build gates.** Transport, implementation, review, and deployment authority need separate controls; a warning learned from one wrong label is more durable than confidence in the next label; and a specification is separated from a wish by a negative test, a regression assertion, and a guard against its author's known unreliability — whoever, and whatever, the author is.

This paper is a direct continuation of **"Recall, Negation, Abstention: Evaluating a Production LLM Agent Against Its Own Failure Modes"** *(Rajaii Labs, July 2026) and* **"Do Not Trust My Labels"** *(Rajaii Labs, July 2026), and references both by section number throughout.*

```
Substrate is a production system of Rajaii Labs, built on Claude.
Architecture and methodology questions welcome: ramin@raminrajaii.com.

© 2026 Ramin Rajaii. All rights reserved.
```

References

Engineering claims are cited inline to the system's own stores, logs, schema, and commit record, per the series convention, and do not appear here. In-text superscripts refer to the entries below, numbered alphabetically.

1. Birch, J. (2024). *The Edge of Sentience: Risk and Precaution in Humans, Other Animals, and AI*. Oxford University Press.
2. Block, N. (1995). On a confusion about a function of consciousness. *Behavioral and Brain Sciences*, 18(2).
3. Butlin, P., Long, R., Elmoznino, E., Bengio, Y., Birch, J., Constant, A., Deane, G., Fleming, S. M., Frith, C., Ji, X., Kanai, R., Klein, C., Lindsay, G., Michel, M., Mudrik, L., Peters, M. A. K., Schwitzgebel, E., Simon, J., & VanRullen, R. (2023). Consciousness in artificial intelligence: Insights from the science of consciousness. arXiv:2308.08708.
4. Butlin, P., Long, R., Bayne, T., Bengio, Y., Birch, J., Chalmers, D., Constant, A., Deane, G., Elmoznino, E., Fleming, S. M., Ji, X., Kanai, R., Klein, C., Lindsay, G., Michel, M., Mudrik, L., Peters, M. A. K., Schwitzgebel, E., Simon, J., & VanRullen, R. (2026). Identifying indicators of consciousness in AI systems. *Trends in Cognitive Sciences*. https://doi.org/10.1016/j.tics.2025.10.011
5. Chalmers, D. J. (2023). Could a large language model be conscious? *Boston Review*.
6. Clark, A. (2013). Whatever next? Predictive brains, situated agents, and the future of cognitive science. *Behavioral and Brain Sciences*, 36(3).
7. Dehaene, S., & Changeux, J.-P. (2011). Experimental and theoretical approaches to conscious processing. *Neuron*, 70(2).
8. Graziano, M. S. A. (2013). *Consciousness and the Social Brain*. Oxford University Press.
9. Locke, J. (1689). *An Essay Concerning Human Understanding*, Book II, Chapter 27.
10. Nagel, T. (1974). What is it like to be a bat? *The Philosophical Review*, 83(4).
11. Nisbett, R. E., & Wilson, T. D. (1977). Telling more than we can know: Verbal reports on mental processes. *Psychological Review*, 84(3).
12. Parfit, D. (1984). *Reasons and Persons*. Oxford University Press.
13. Schwitzgebel, E. (2008). The unreliability of naive introspection. *The Philosophical Review*, 117(2).
14. Tulving, E. (1985). Memory and consciousness. *Canadian Psychology*, 26(1).

15. Wilterson, A. I., & Graziano, M. S. A. (2021). The attention schema theory in a neural network agent: Controlling visuospatial attention using a descriptive model of attention. *Proceedings of the National Academy of Sciences*, 118(33).